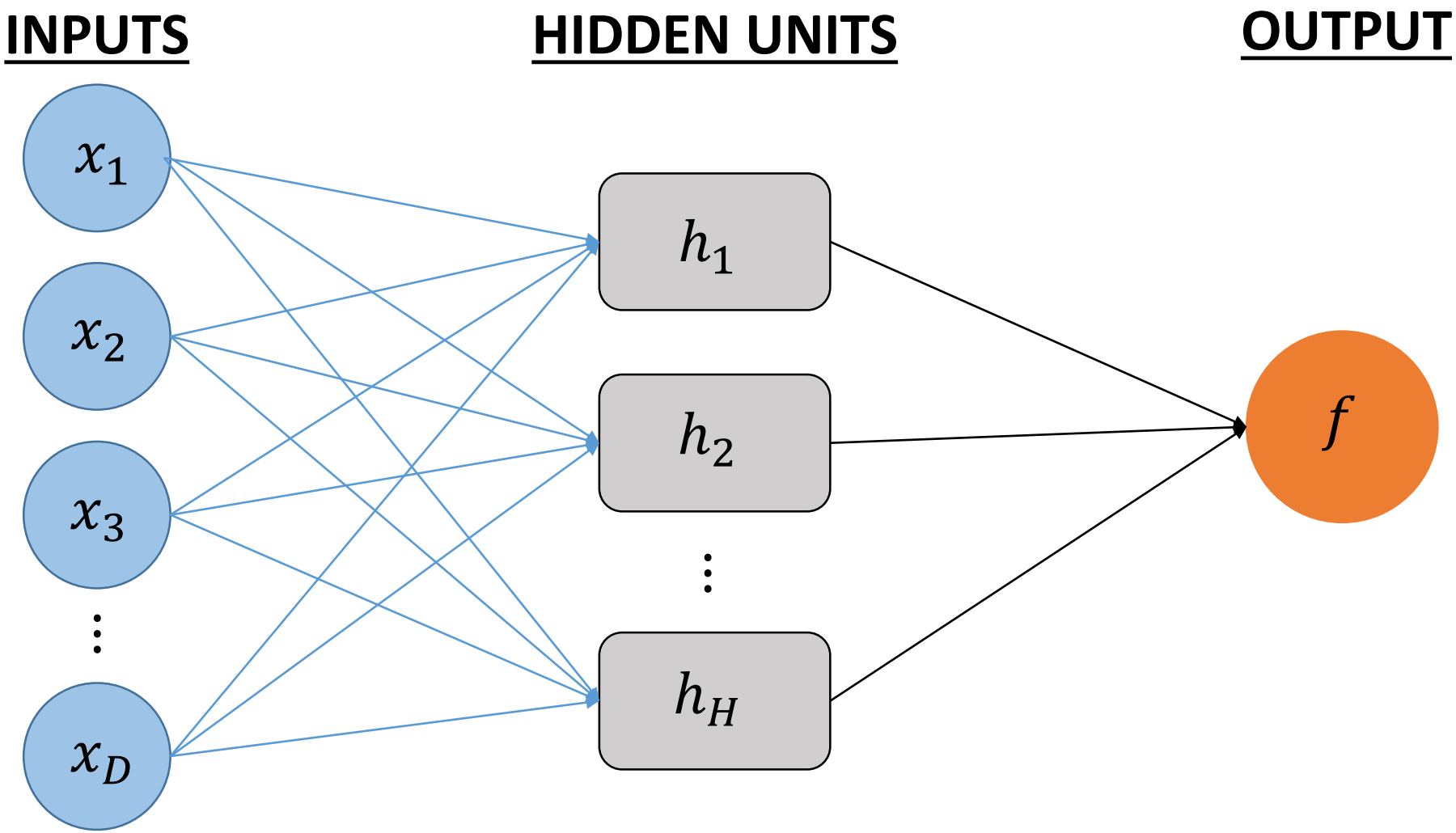
INFSCI 2595: Machine Learning

Week 14: Neural Networks

Spring 2024

So far we have focused on neural networks for regression tasks



We discussed how a neural network is essentially a series of matrix operations!

- Bind all hidden unit parameters into a matrix: $\mathbf{B} = [\boldsymbol{\beta}_1 \quad \boldsymbol{\beta}_2 \quad \cdots \quad \boldsymbol{\beta}_H]$
- All hidden unit "linear predictors" are equal to:

$$\mathbf{A} = \mathbf{XB}$$

- All hidden unit non-linear values are then:

$$\mathbf{H} = g(\mathbf{A}) = g(\mathbf{XB})$$

- Neural network response calculated with output layer intercept (bias) α_0 and column vector of "weights" $\boldsymbol{\alpha}$:

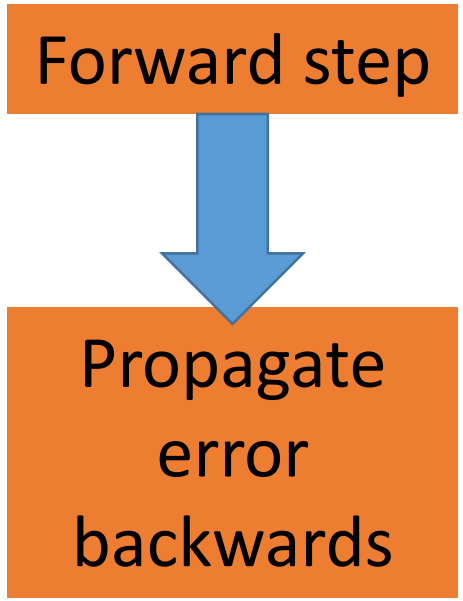
$$\mathbf{f} = \alpha_0 + \mathbf{H}\boldsymbol{\alpha} = \alpha_0 + [g(\mathbf{XB})]\boldsymbol{\alpha}$$

We derived the key expressions of the backpropagation method for calculating the gradient

The backpropagation method is:

- Given input design matrix, $\mathbf{X}$, and current guess for $\boldsymbol{\theta}$:
  - Calculate all hidden units, $\mathbf{H}$
  - Calculate the neural network response, $\mathbf{f}$

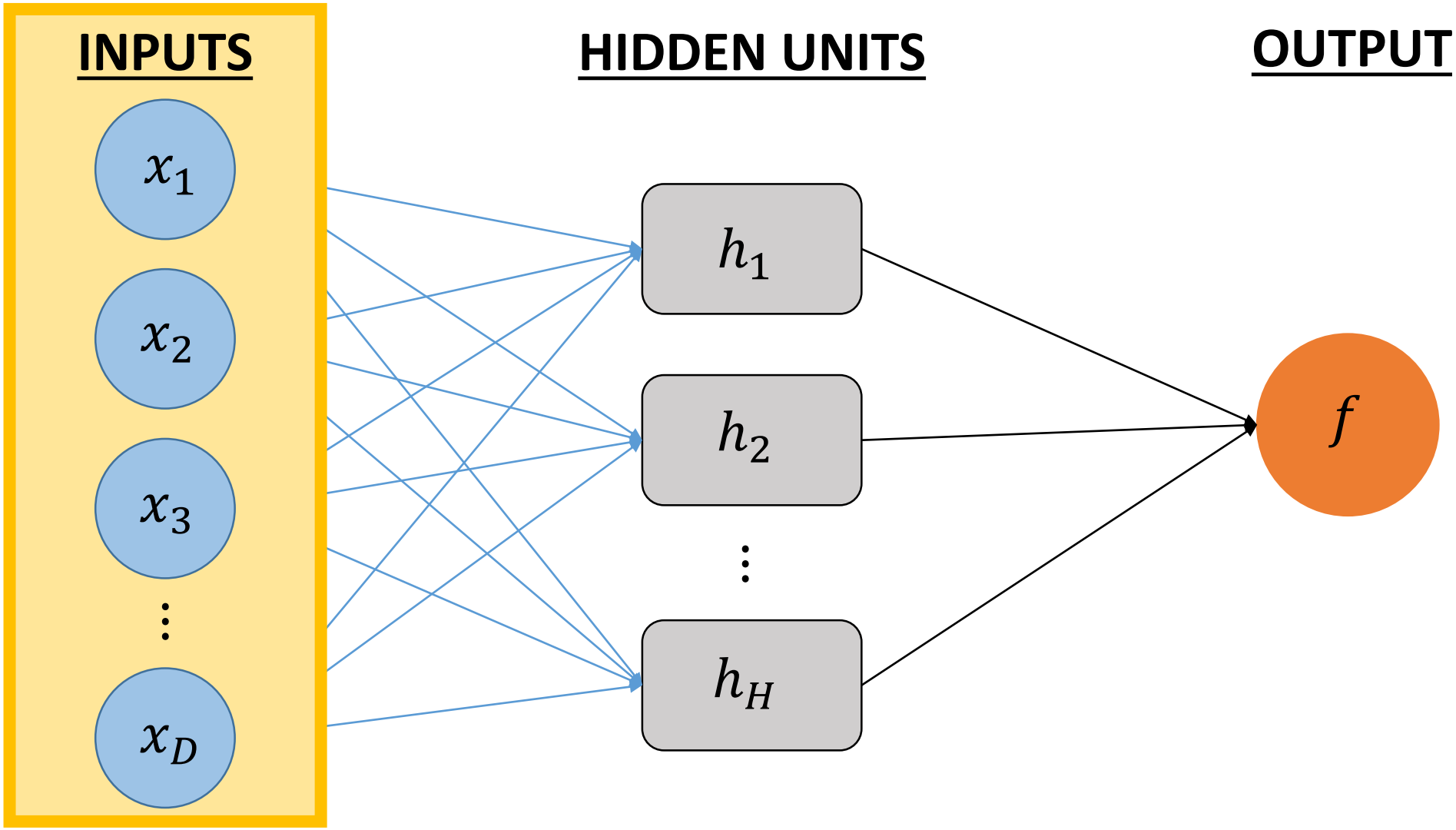
- Given $\mathbf{y}$ and $\mathbf{f}$, calculate the deltas, $\boldsymbol{\delta}$
- Backpropagate $\{\delta_n\}_{n=1}^{N}$ to calculate all $\left\{\{\xi_{n,k}\}_{k=1}^{H}\right\}_{n=1}^{N}$
- Calculate all derivatives



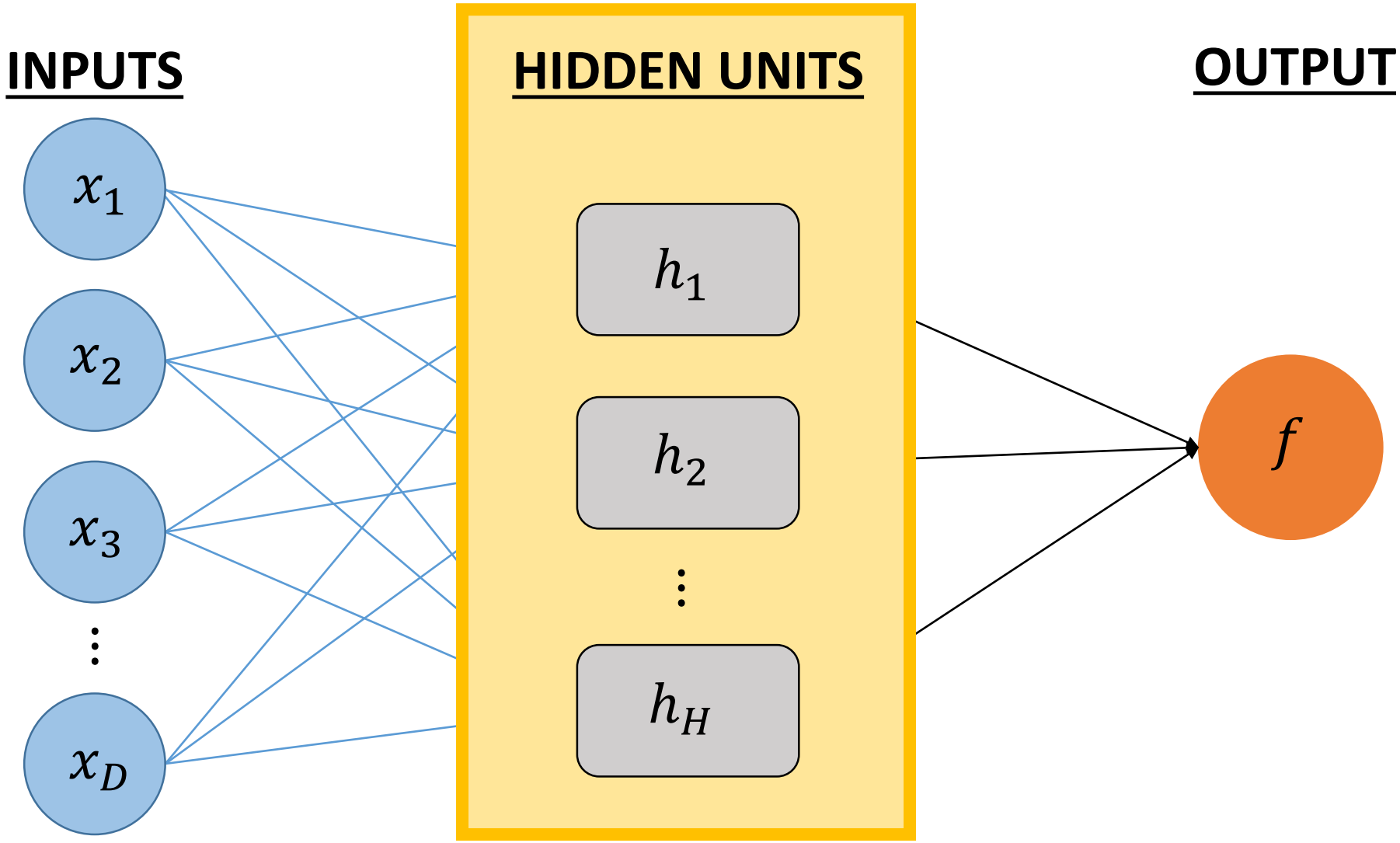
What if we want to use the neural network for binary classification instead of regression?

- What would need to change in the single layer neural network?

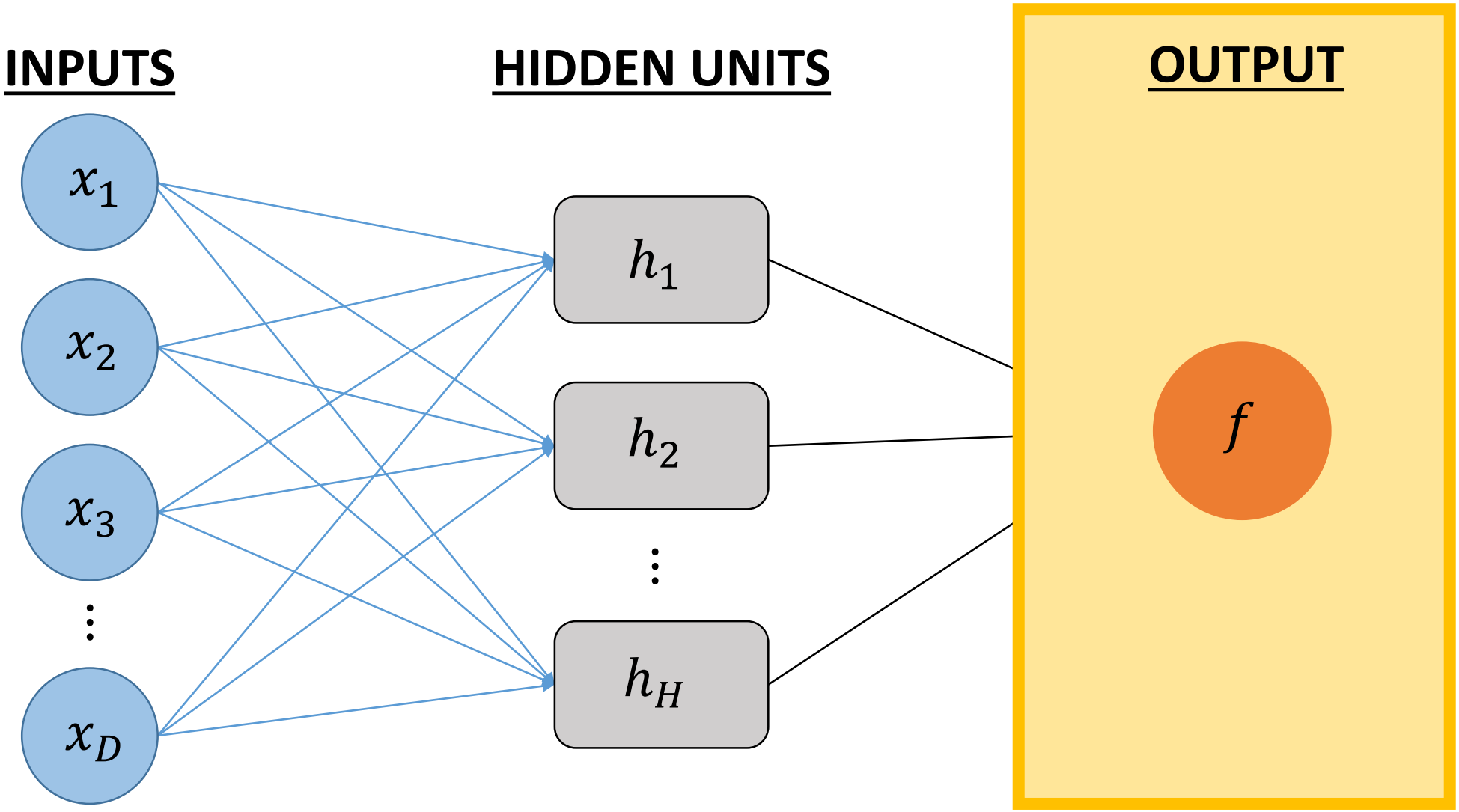
Would we change the inputs?



Would we change the hidden units?



What if we changed the output layer…



What if we changed the output layer…

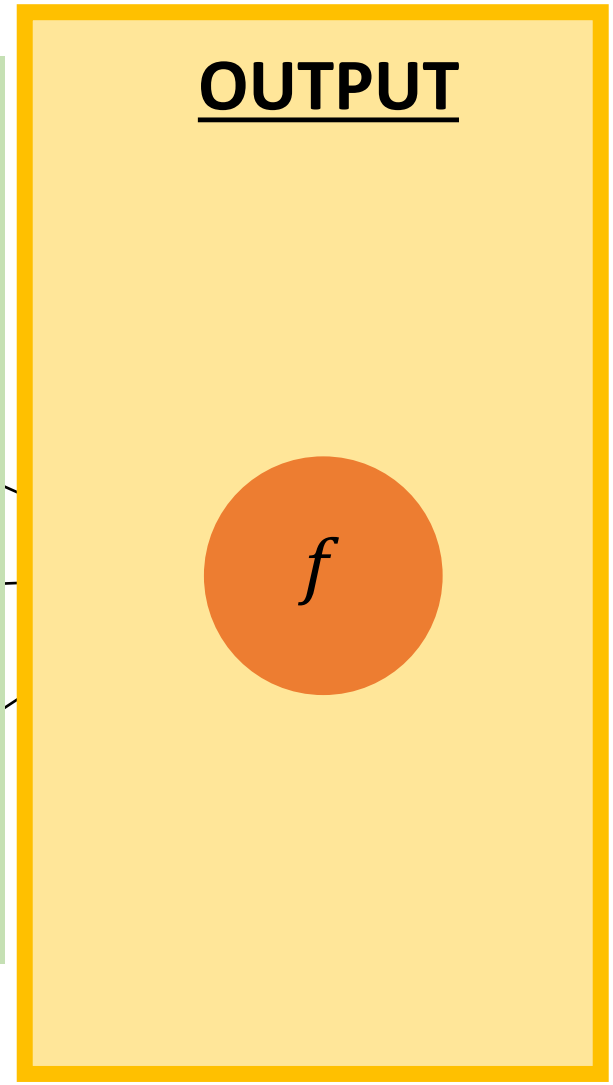
Think back to logistic regression, we wanted to model the **EVENT PROBABILITY**, μ.

To accomplish that, we applied a linear model to a transformed unbounded variable:

$$\eta_n = \sum_{d=0}^{D} (\beta_d x_n)$$

We recovered μ_n by **back-transforming**:

$$\mu_n = \text{logit}^{-1}(\eta_n)$$



We will follow the same strategy for the neural network

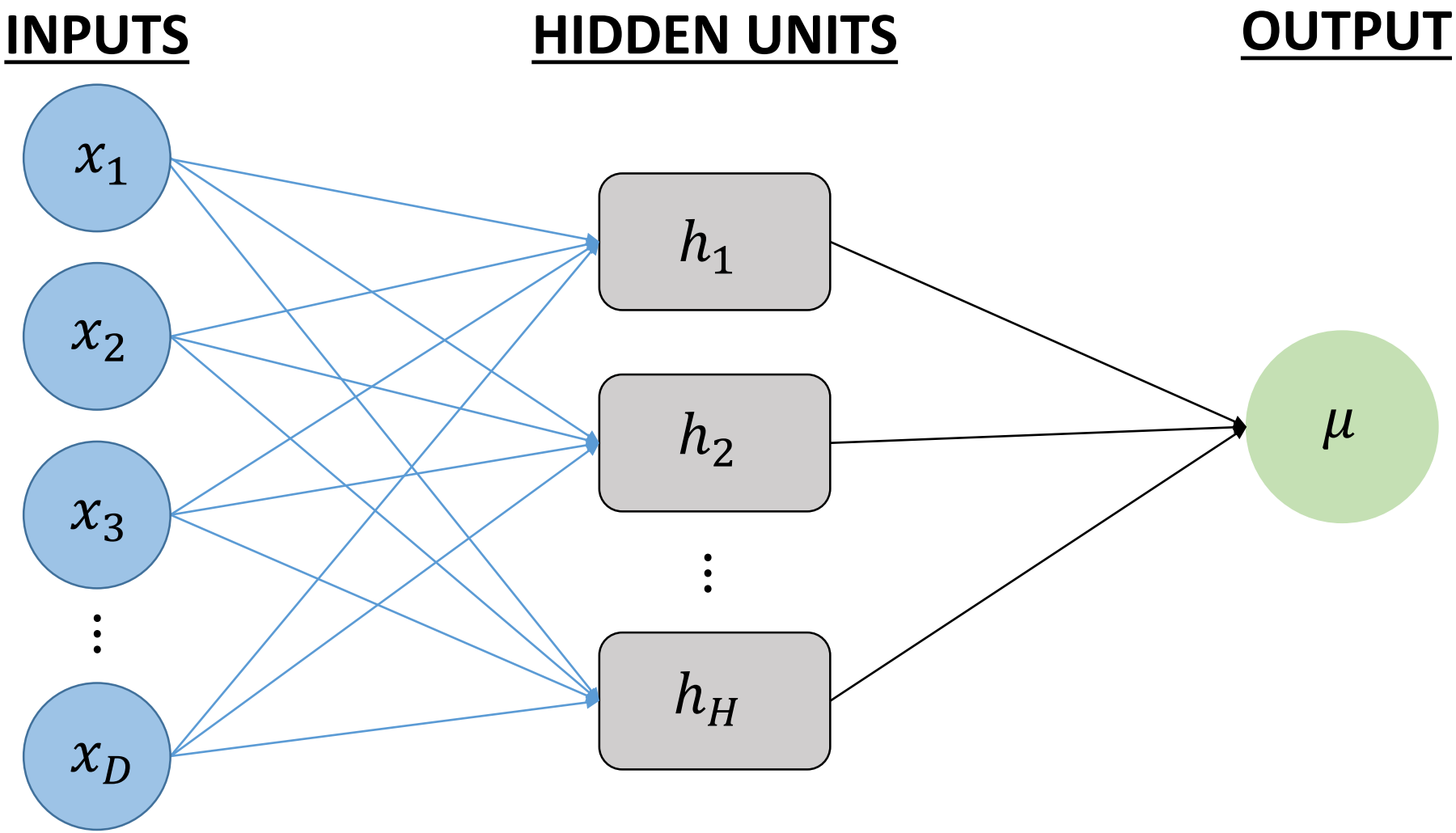
- Apply the logistic function as a non-linear transformation to the output layer:

$$\mu_n = \text{logit}^{-1}\left(\sum_{k=0}^{H}\left(\alpha_k h_{n,k}\right)\right)$$

- Or in matrix form:

$$\boldsymbol{\mu} = \text{logit}^{-1}(\alpha_0 + \mathbf{H}\boldsymbol{\alpha})$$

The binary classification network…therefore looks a lot like the regression network!



What else needs to change? How will we fit the neural network?

What else needs to change? How will we fit the neural network?

- We need a different loss function.

- Instead of the SSE the conventional loss function for binary classification is the binary **cross-entropy** loss function:

$$-\sum_{n=1}^{N}(y_n \log[\mu_n] + (1 - y_n) \log[1 - \mu_n])$$

What else needs to change? How will we fit the neural network?

- We need a different loss function.

- Instead of the SSE the conventional loss function for binary classification is the binary **cross-entropy** loss function:

$$-\sum_{n=1}^{N}(y_n \log[\mu_n] + (1-y_n)\log[1-\mu_n])$$

Where have you seen this before?!?!

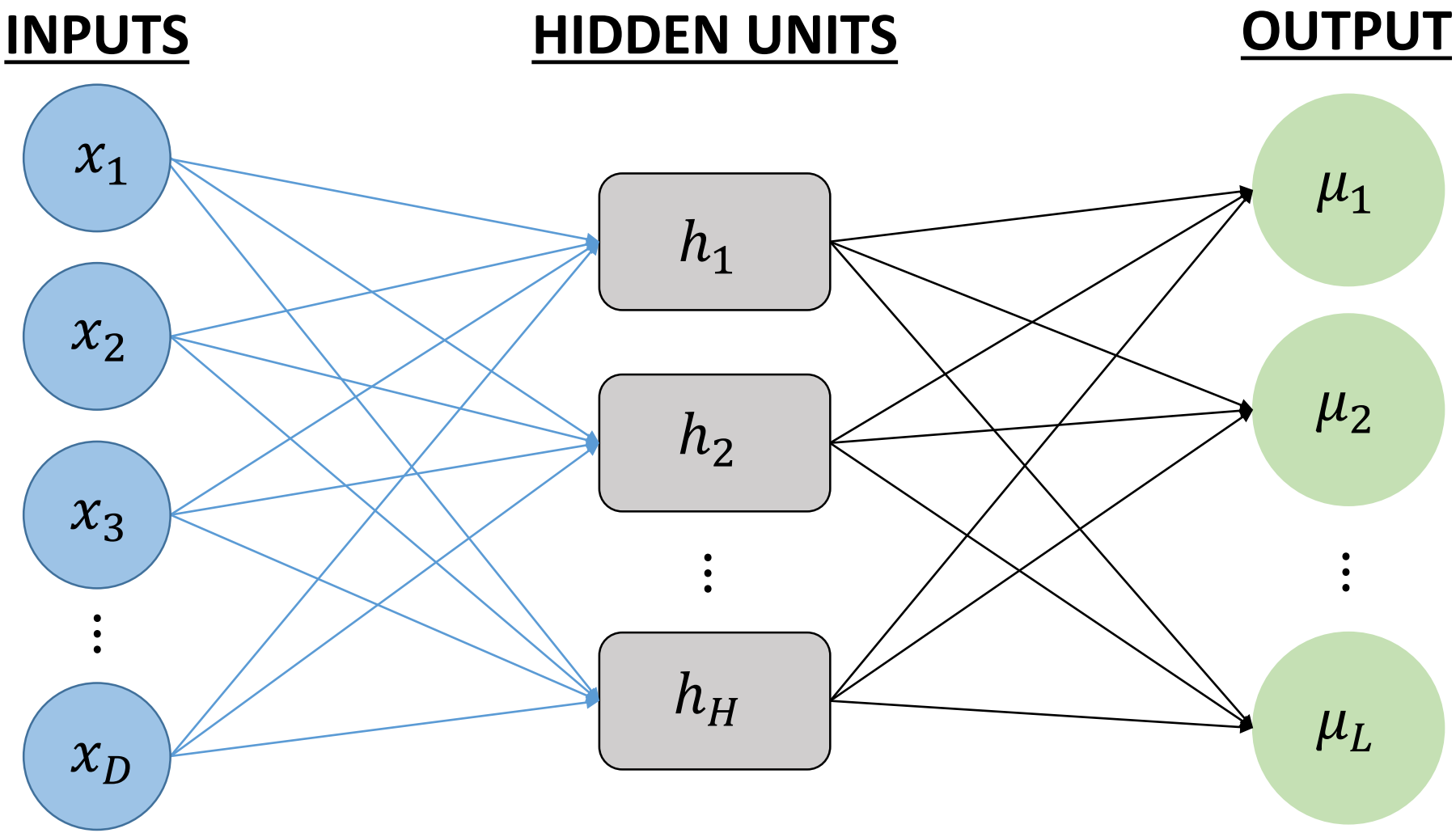
Minimizing the binary cross-entropy loss function is **equivalent** to **maximizing** the **Bernoulli likelihood**!

$$\log[\text{Bernoulli}(y_n \mid \mu_n)] = y_n \log[\mu_n] + (1 - y_n) \log[1 - \mu_n]$$

The backpropagation method can still used to evaluate the gradient

- However, the expressions will be different from the regression task.
- **The output layer must now account for the output layer transformation**.
- Also, the likelihood function has changed, which will also modify the final expressions.
- The concepts however are the same, back-propagate the output error to the hidden layer.

What if we have $L > 2$ output classes…



We need to ensure the output layer probabilities sum to 1.

Each output class has its own set of parameters:

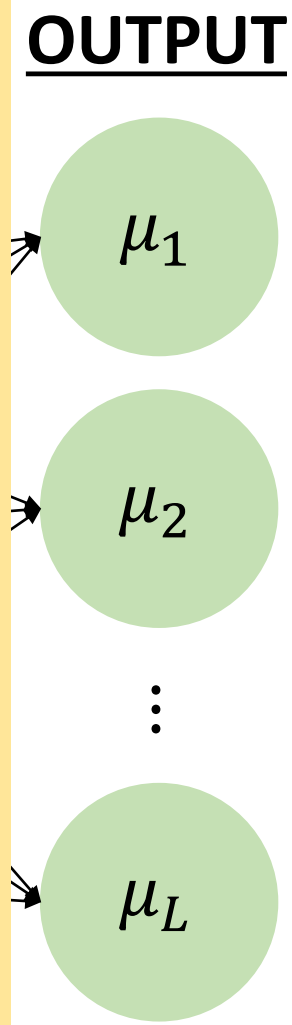
$$\{\alpha_{0,1}, \boldsymbol{\alpha}_1\}, \{\alpha_{0,2}, \boldsymbol{\alpha}_2\}, \dots, \{\alpha_{0,l}, \boldsymbol{\alpha}_l\}, \dots, \{\alpha_{0,L}, \boldsymbol{\alpha}_L\}$$

The **SOFTMAX** function is applied as the non-linear function to the output layer.

The l-th class probability is equal to:

$$\mu_{n,l} = \frac{\exp\left(\alpha_{0,l} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_l\right)}{\sum_{l=1}^{L}\left(\exp\left(\alpha_{0,l} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_l\right)\right)}$$

Ensures that $\sum_{l=1}^{L} \mu_{n,l} = 1$.



Softmax function for $L = 3$ classes

- The Softmax function applied to the first class in the output layer:

$$\mu_{n,l=1} = \frac{\exp\left(\alpha_{0,1} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_1\right)}{\exp\left(\alpha_{0,1} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_1\right) + \exp\left(\alpha_{0,2} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_2\right) + \exp\left(\alpha_{0,3} + \mathbf{h}_{n,:}\boldsymbol{\alpha}_3\right)}$$

- The Softmax function guarantees that each class has an output between 0 and 1 and the sum across all classes equals 1.

Loss function for the multi-class neural network classifier

- The **cross-entropy loss function**:

$$-\sum_{n=1}^{N}\left(\sum_{l=1}^{L}\left(y_{n,l}\log[\mu_{n,l}]\right)\right)$$

- Where $y_{n,l}=1$ if the n-th observation corresponds to the l-th class, and $y_{n,l}=0$ otherwise.

For $L = 3$ classes

- The cross-entropy loss function:

$$-\sum_{n=1}^{N}\left(y_{n,l=1}\log\left[\mu_{n,l=1}\right]+y_{n,l=2}\log\left[\mu_{n,l=2}\right]+y_{n,l=3}\log\left[\mu_{n,l=3}\right]\right)$$

- Where $y_{n,l} = 1$ if the n-th observation corresponds to the l-th class, and $y_{n,l} = 0$ otherwise.

If $L = 2$…there are just two classes…

- If $y_{n,l=1} = 1$ then $y_{n,l=2}$ must equal 0.
- Or, we can write: $y_{n,l=2} = 1 - y_{n,l=1}$.
- If the probability of class 1 is $\mu_{n,l=1}$, then the probability of class 2 must be: $\mu_{n,l=2} = 1 - \mu_{n,l=1}$

If $L = 2$…there are just two classes…

- The cross-entropy loss function:

$$-\sum_{n=1}^{N}\left(y_{n,l=1}\log\left[\mu_{n,l=1}\right]+y_{n,l=2}\log\left[\mu_{n,l=2}\right]\right)$$

- Simplifies to the binary cross-entropy function!

$$-\sum_{n=1}^{N}\left(y_{n,l=1}\log\left[\mu_{n,l=1}\right]+\left(1-y_{n,l=1}\right)\log\left[1-\mu_{n,l=1}\right]\right)$$